INF1007 — INTRODUCTION À LA PROGRAMMATION · A2026

Projet 2 : de la donnée à l'application

Entraîner un modèle simple, puis le mettre à la disposition d'un utilisateur grâce à une interface graphique.

ÉQUIPE	DURÉE	PRÉSENTATION	REMISE
2 étudiants	≈ 8 h de travail	10 min (5 + 5)	Sur Moodle

1 Objectif du projet

En équipe de deux, vous choisissez un jeu de données dans un domaine qui vous intéresse, vous entraînez un modèle simple qui **classe** ou **prédit** quelque chose, puis vous développez une application dotée d'une **interface graphique** qui permet à un utilisateur d'interroger ce modèle.

Le projet fait appel à tout ce que vous avez vu aux chapitres 5 à 13 : fichiers, fonctions, modules, tests, programmation orientée objet, NumPy et Matplotlib. Il vous demande aussi d'aller chercher par vous-mêmes trois outils que le cours ne détaille pas (**pandas**, **scikit-learn** et une **bibliothèque d'interface graphique**) en vous appuyant sur leur documentation officielle, comme au chapitre 13.

VOLET 1
Jeu de données
Choisir un domaine et un jeu de données (Kaggle ou autre source ouverte) qui respecte les critères de la section 2.

VOLET 2
Modèle
Préparer les données, entraîner un modèle simple, l'évaluer honnêtement et le sauvegarder (section 3).

VOLET 3
Interface graphique
Développer une application qui charge le modèle, valide les saisies, affiche la prédiction et un graphique (section 4).

> **Ce qui compte**
> On évalue votre démarche et la qualité de votre travail, **pas la performance du modèle**. Un résultat modeste, bien analysé et bien justifié vaut mieux qu'un score élevé que vous ne savez pas expliquer.

Travail en équipe et validations

- **Binôme.** Les deux membres écrivent du code (l'historique Git le montrera) et doivent pouvoir expliquer *tout* le projet. Pratiquez la programmation en paire : un pilote qui écrit, un copilote qui relit, et changez de rôle régulièrement.
- **Validations.** Après chaque choix important (domaine, jeu de données, modèle, interface), nous vous recommandons fortement d'en discuter avec votre auxiliaire d'enseignement et de **justifier votre choix** : qu'avez-vous envisagé d'autre, et pourquoi celui-ci ? Les points de validation sont détaillés à la section 6.
- **Outils d'IA.** Leur usage est permis à condition d'être déclaré dans un journal (section 7). Vous restez responsables de chaque ligne remise.

2 Volet 1 : choisir un jeu de données

Le domaine est libre (santé, sport, environnement, immobilier, transport, éducation, loisirs, etc.). Le jeu de données peut venir de Kaggle ou de toute autre source ouverte : l'annexe A propose des pistes.

Critères obligatoires

Critère	Exigence
Format	Données tabulaires dans un fichier CSV (ou convertible en CSV) : une ligne par observation, une colonne par variable.
Taille	Entre 500 et 50 000 lignes, avec au moins 5 colonnes d'entrée (les « caractéristiques »).
Variable cible	Une colonne cible claire : soit une catégorie à prédire (*classification*), soit une valeur numérique à estimer (*régression*).
Types de variables	Parmi les caractéristiques, au moins une variable **numérique** et au moins une variable **catégorielle** : votre interface devra gérer les deux types de saisie.
Source et licence	Licence compatible avec un usage éducatif, source citée dans le README.
Données personnelles	Aucune donnée permettant d'identifier une personne (nom, adresse, courriel, etc.).
Jeux exclus	Iris, Titanic et MNIST : leurs solutions sont partout.

À présenter à votre auxiliaire (validations A et B)

- **Le domaine :** pourquoi celui-là ? Que pourrait faire un utilisateur avec votre application ?
- **Le jeu de données :** pourquoi celui-là plutôt qu'un autre ? Comment avez-vous vérifié chaque critère (lignes, types de colonnes, licence) ?

3 Volet 2 : entraîner un modèle simple

Le modèle doit rester simple : régression linéaire ou logistique, k plus proches voisins, arbre de décision ou forêt aléatoire. L'apprentissage profond est exclu. **Un seul modèle suffit** ; en comparer deux est un plus, mais ne vous éloignez pas de l'interface.

#	Étape	Ce qui est attendu
1	**Explorer et nettoyer**	Charger les données avec pandas ; examiner les types, les valeurs manquantes et les valeurs aberrantes ; expliquer en quelques lignes ce que vous avez corrigé et pourquoi. Produire au moins un graphique d'exploration.
2	**Définir le problème**	Variable cible, type de problème (classification ou régression) et **métrique d'évaluation justifiée** (exactitude, F1, RMSE, R^2, etc.).
3	**Séparer les données**	Ensembles d'entraînement et de test (`train_test_split`) avec une graine aléatoire fixée pour que les résultats soient reproductibles. On n'évalue jamais un modèle sur les données qui ont servi à l'entraîner.
4	**Établir une référence**	Un modèle trivial (classe majoritaire, moyenne) qui sert de point de comparaison : sans lui, impossible de dire si votre score est bon.

#	Étape	Ce qui est attendu
5	**Entraîner et évaluer**	Entraîner le modèle choisi, calculer la métrique sur l'ensemble de test, la comparer à la référence, puis commenter honnêtement les résultats et les limites (par exemple avec une matrice de confusion ou un graphique « prédit contre réel »).
6	**Sauvegarder**	Enregistrer le modèle entraîné (avec `joblib`, installé avec scikit-learn) et un **fichier JSON de métadonnées** : noms des colonnes, valeurs possibles des variables catégorielles, intervalles des variables numériques, métrique obtenue. L'interface s'en servira pour se construire : le modèle n'est *pas* ré-entraîné à chaque lancement.

Validation C À présenter avant d'entraîner : la variable cible, la métrique choisie et le modèle retenu, avec les raisons.

4 Volet 3 : développer l'interface graphique

La technologie est **libre** (Tkinter, PyQt ou PySide, Streamlit, Gradio, etc.), mais elle doit être **approuvée par votre auxiliaire** (validation D) et fonctionner en local. Elle est décrite dans le README, avec les commandes pour installer et lancer l'application.

Validation D À présenter avant de coder l'interface : la technologie choisie et une **maquette sur papier** de la fenêtre principale.

Fonctionnalités minimales

Fonctionnalité	Ce qui est attendu
Saisie des caractéristiques	Un champ par variable d'entrée, adapté à son type (champ numérique, liste déroulante pour une catégorie, etc.), construit à partir des métadonnées du modèle sauvegardé.
Validation des saisies	Une saisie vide, non numérique ou hors intervalle produit un **message clair**, jamais un plantage ni un traceback.
Prédiction	Un bouton « Prédire » affiche le résultat dans un langage compréhensible par un non-informaticien, avec une probabilité ou un indice de confiance si le modèle le permet.
Graphique intégré	Au moins un graphique Matplotlib *dans* l'interface (importance des variables, position de la prédiction dans la distribution de la cible, matrice de confusion, prédit contre réel, etc.), avec titre, axes étiquetés et légende si utile.
Historique	Chaque prédiction est ajoutée à un fichier (CSV ou JSON) et peut être consultée depuis l'interface.
Avertissement	Si le domaine est sensible (santé, finance, emploi, justice), l'interface précise que l'outil est un exercice pédagogique et non une aide à la décision. Le Therac-25 (chapitre 8) rappelle les risques d'une confiance aveugle.

5 Exigences d'implémentation, chapitre par chapitre

Ces exigences s'appliquent quelle que soit la technologie d'interface : **l'interface appelle la logique du projet, elle ne la contient pas** (« séparer le calcul et les entrées-sorties », chapitre 8). C'est ce qui rend la logique testable.

Chapitre	Exigences
Ch. 5 Entrées et sorties	• Lire le jeu de données (CSV) ; écrire les métadonnées du modèle (JSON) et l'historique des prédictions (CSV ou JSON). • Construire les chemins avec le module `os`, vérifier qu'un fichier existe avant de l'ouvrir et utiliser `with` pour chaque ouverture.
Ch. 6 Fonctions	• Fonctions courtes, à responsabilité unique, bien nommées. Chacune a une **docstring** (ce qu'elle reçoit, ce qu'elle renvoie, dans quel cas elle échoue) et des **annotations de type**. • Les fonctions de calcul retournent leur résultat avec `return` ; l'affichage est ailleurs.
Ch. 7 Fonctions avancées et modules	• Code réparti en **au moins 4 modules** (par exemple données, modèle, exceptions, interface) qui s'importent les uns les autres, plus un point d'entrée `main.py`. • Au moins un outil du chapitre utilisé là où il a du sens : décorateur (chronométrer l'entraînement), générateur (lire le CSV par blocs), `lambda` (clé de tri). À expliquer en une phrase dans le README.
Ch. 8 Exceptions, tests, Git et CI	• **Exceptions :** `try` / `except` sur des exceptions précises (jamais un `except` nu) et au moins une **exception personnalisée** levée avec `raise` (par exemple `DonneeInvalideError`, qui hérite de `Exception`). • **Tests :** au moins **8 tests** `pytest` sur la logique (hors interface), couvrant cas usuels, cas limites et cas d'erreur (`pytest.raises`), avec `pytest.approx` pour les flottants. Si une IA génère des tests, relisez-les : un test écrit d'après le code ne prouve pas grand-chose. • **Couverture :** la mesurer avec `coverage` et indiquer le résultat dans le README. Aucun seuil n'est exigé. • **Git et CI :** dépôt GitHub public et partagé, commits réguliers des **deux** membres avec des messages clairs, `requirements.txt` et un workflow GitHub Actions qui lance `pytest` à chaque `push`. Le dernier commit doit passer la CI (marque verte).
Ch. 9 à 11 Programmation orientée objet	• **Au moins 3 classes** (par exemple `JeuDeDonnees`, `Modele`, `Application`), chacune avec un constructeur, des méthodes qui agissent sur son état et des attributs protégés ou privés quand c'est pertinent (encapsulation). • **Héritage et polymorphisme :** une classe parente (idéalement abstraite) et **au moins deux classes enfants** qui redéfinissent la même méthode. Exemple naturel avec vos données : `Variable` → `VariableNumerique` et `VariableCategorielle`, chacune validant sa saisie à sa façon. • **Composition :** au moins une classe qui contient des instances d'autres classes (par exemple l'application contient le modèle et la liste des variables). • Au moins une méthode spéciale redéfinie (`__str__`, `__repr__`, `__eq__`, etc.).
Ch. 12 NumPy	• Au moins un calcul **vectorisé** (sans boucle Python) dans la logique du projet. Idée : calculer vous-mêmes la métrique choisie avec NumPy et vérifier, dans un test, qu'elle coïncide avec celle de scikit-learn.

Chapitre	Exigences
Ch. 13 Matplotlib, documentation et IA	• **Au moins deux graphiques lisibles** : un d'exploration (enregistré avec `savefig` et montré à la présentation) et un dans l'interface. • **Documentation officielle :** citer dans le README au moins trois pages consultées (pandas, scikit-learn, technologie d'interface). • **Journal d'utilisation de l'IA** (section 7) : une réponse d'IA peut être fausse avec aplomb, on la vérifie dans la documentation ou par un test.

6 Déroulement : étapes, temps et validations

Les durées sont **indicatives** et totalisent 8 h de travail par équipe, préparation de la présentation comprise. Écrivez vos tests et faites vos commits *au fil du travail*, pas à la fin.

#	Étape	Contenu	Durée	Validation
0	**Mise en place**	Créer le dépôt GitHub **public**, y inviter votre coéquipier, préparer la structure de dossiers, le README, `requirements.txt` et un workflow CI avec un premier test qui passe.	0 h 30	
1	**Domaine et jeu de données**	Explorer deux ou trois pistes, vérifier les critères de la section 2, charger le jeu retenu avec pandas et produire un premier graphique.	1 h 30	A domaine B jeu de données
2	**Préparation et modèle**	Nettoyage, séparation, référence, entraînement, évaluation et sauvegarde (section 3), avec les modules de données et de modèle et leurs premiers tests.	2 h 00	C cible, métrique et modèle
3	**Interface graphique**	Choisir la technologie et dessiner la maquette (10 min), puis coder les classes de variables, l'application, le graphique et l'historique.	2 h 15	D technologie et maquette
4	**Consolidation**	Tests complémentaires et couverture, docstrings, README, fiche de décisions, journal d'IA. Cloner le dépôt dans un dossier neuf pour vérifier que tout s'installe et se lance, et que la CI est verte.	1 h 15	
5	**Préparation de la présentation**	Répartir la parole, préparer la démo et le plan B, répéter en chronométrant (5 minutes).	0 h 30	
	Total		**8 h 00**	

Comment se passe une validation (5 minutes)

Présentez à l'auxiliaire **ce que vous avez choisi**, **ce que vous avez envisagé d'autre** et **pourquoi**. Notez la décision et la justification dans votre fiche de décisions (section 7), avec la date. Si l'auxiliaire vous suggère de changer d'avis, vous restez libres, mais notez pourquoi vous maintenez ou modifiez votre choix.

7 Livrables

Livrable	Contenu
Dépôt GitHub code, tests, CI	• Dépôt **public** ; son lien est remis sur Moodle, dans la boîte de dépôt correspondante. • `README.md` : description du projet, source et licence du jeu de données, commandes d'installation et de lancement, structure du code, résultat du modèle (métrique et référence), couverture de tests, références à la documentation, outil du chapitre 7 utilisé. • Le jeu de données dans `donnees/` s'il fait moins de 25 Mo et si sa licence permet de le redistribuer (le dépôt est public) ; sinon un lien et les instructions de téléchargement dans le README.
Fiche de décisions `DECISIONS.md`, 1 à 2 pages	Un tableau rempli *au fil du projet*, une ligne par choix : domaine, jeu de données, cible et métrique, modèle, technologie d'interface, et tout autre choix important. Colonnes : Choix · Options envisagées · Choix retenu et justification · Validé par l'auxiliaire (date).
Journal d'utilisation de l'IA `JOURNAL_IA.md`	Au moins 5 entrées, même si vous n'avez presque pas utilisé d'IA (dans ce cas, dites-le). Au moins une entrée montre une vérification qui a changé votre décision. Colonnes : Outil · Ce que vous avez demandé · Réponse reçue · Vérification faite (documentation, test, exécution) · Décision (gardé, modifié, rejeté).

Remise : sur Moodle, dans la boîte correspondante, avec le **lien vers votre dépôt GitHub public**.
Date limite : _____ Date des présentations : _____

8 Présentation orale : 10 minutes

5 minutes d'exposé, suivies de **5 minutes de questions**. Les deux membres parlent à parts à peu près égales. Les diapositives sont facultatives (4 au maximum) ; la démo en direct est obligatoire.

Minutage	Partie	Contenu suggéré
0:00 – 0:45	**Problème et données**	Le domaine, ce que fait l'application, pourquoi ce jeu de données.
0:45 – 1:45	**Modèle**	Cible, modèle retenu, métrique, score comparé à la référence, limites.
1:45 – 3:15	**Démo en direct**	Un cas normal, une saisie invalide et le graphique de l'interface.
3:15 – 4:15	**Conception**	Un schéma simple des classes et des modules, et un choix de conception que vous défendez.
4:15 – 5:00	**Bilan**	Difficultés rencontrées, ce que vous referiez autrement, rôle de l'IA dans le projet.

- **Prévoyez un plan B pour la démo** (captures d'écran ou courte vidéo) et lancez l'application avant de commencer.
- **Questions :** toute ligne de code est une cible possible (expliquer une fonction, justifier un choix de la fiche de décisions, prédire un comportement), adressée à l'un ou l'autre membre.

9 Évaluation (barème indicatif)

Critère	Points	On regarde surtout
Jeu de données et problème	10	Critères respectés, pertinence du domaine, cible et métrique clairement définies.
Modèle	15	Préparation des données, séparation entraînement/test, référence, évaluation honnête, analyse des résultats et des limites.
Interface graphique	20	Fonctionnalités minimales, validation des saisies (aucun plantage), graphique intégré, clarté pour un utilisateur non informaticien.
Conception et qualité du code	20	Modules, POO (héritage, composition, encapsulation), docstrings et annotations, exceptions, NumPy, outil du chapitre 7 pertinent.
Tests, Git et CI	10	Pertinence des tests (usuel, limite, erreur), CI verte, historique Git qui reflète le travail des deux membres.
Justification des choix et usage de l'IA	10	Fiche de décisions complète, validations avec l'auxiliaire, journal d'IA honnête et critique, références à la documentation.
Présentation et questions	15	Clarté, respect du temps, démo, capacité de *chaque* membre à expliquer le code.
Total	**100**	

La note peut être ajustée individuellement selon l'historique Git et les réponses aux questions.

Annexes

A. Où trouver un jeu de données

- **Kaggle** (kaggle.com/datasets) : compte gratuit requis ; filtrez par format CSV.
- **UCI Machine Learning Repository** (archive.ics.uci.edu).
- **OpenML** (openml.org).
- **Google Dataset Search** (datasetsearch.research.google.com).
- **Données Québec** (donneesquebec.ca).
- **Gouvernement du Canada** (ouvert.canada.ca).
- **Statistique Canada** (statcan.gc.ca).
- **data.gouv.fr** (données ouvertes de la France).

Lisez la licence indiquée sur la page du jeu de données avant de le retenir.

Idées de domaines, pour vous inspirer (ce ne sont pas des jeux de données imposés) :

Domaine	Exemple de question à laquelle l'application répond
Immobilier	Quel est le prix probable d'un logement selon sa superficie, son quartier et son âge ?
Transport	Un vol sera-t-il en retard ? Quel est le prix probable d'un billet ?
Environnement et énergie	Quelle sera la consommation d'énergie d'un bâtiment ? La qualité de l'air sera-t-elle bonne ?
Sport	Quel sera le résultat d'un match ou la performance d'un athlète selon ses statistiques ?
Culture et loisirs	Quelle note un film ou un jeu vidéo recevra-t-il ? Une chanson sera-t-elle populaire ?

Domaine	Exemple de question à laquelle l'application répond
Santé et bien-être	Quel est le niveau de risque d'après des habitudes de vie ? *Avertissement obligatoire dans l'interface.*

B. Structure de dépôt suggérée

```
projet_inf1007/
├── README.md
├── DECISIONS.md
├── JOURNAL_IA.md
├── requirements.txt
├── .github/workflows/tests.yml     # lance pytest à chaque push
├── donnees/                        # jeu de données (CSV)
├── modeles/                        # modèle sauvegardé + métadonnées JSON
├── src/
│   ├── donnees.py                  # lecture, nettoyage, classes de variables
│   ├── modele.py                   # entraînement, évaluation, prédiction
│   ├── exceptions.py               # exceptions personnalisées
│   ├── interface.py                # interface graphique
│   └── main.py                     # point d'entrée
└── tests/
    ├── test_donnees.py
    └── test_modele.py
```

C. Ressources pour démarrer

Aucun de ces outils n'est enseigné en cours : c'est le moment d'appliquer ce que le chapitre 13 explique sur la lecture d'une documentation et l'usage critique de l'IA.

Outil	Où chercher
pandas	Documentation officielle (pandas.pydata.org), guide d'introduction « 10 minutes to pandas ».
scikit-learn	Documentation officielle (scikit-learn.org), section « Getting started » et guide « Choosing the right estimator ».
Interface graphique	Tkinter : docs.python.org (module `tkinter`). Streamlit : docs.streamlit.io. Gradio : gradio.app. Pour intégrer un graphique Matplotlib dans Tkinter, voir les exemples « user interfaces » de matplotlib.org.
pytest et CI	docs.pytest.org ; docs.github.com (section GitHub Actions) ; chapitre 8 du cours.

D. Pistes facultatives

À envisager seulement si l'essentiel est terminé ; elles ne remplacent aucune exigence.

- Prédictions par lots : charger un CSV dans l'interface et exporter les résultats.
- Comparer un second modèle et expliquer lequel vous gardez.
- Régression linéaire : retrouver les coefficients à la main avec `np.linalg.solve` ou `lstsq` (chapitre 12) et les comparer à ceux de scikit-learn.
- Chapitre 13 : choisir un seuil de décision en optimisant une métrique avec `scipy.optimize`, ou dériver symboliquement une fonction de coût simple avec SymPy et vérifier le résultat numériquement.